COMPUTER ENGINEERING

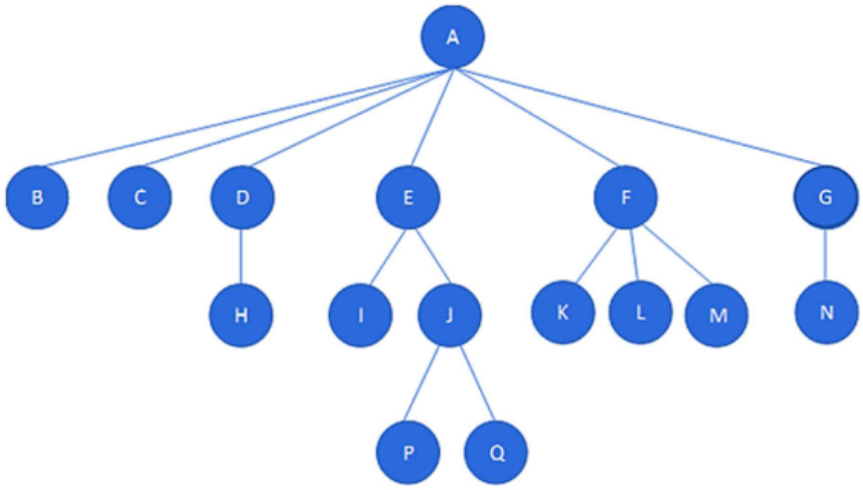
QUEZON CITY

COURSE CODE/TITLE:	CPE-010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	9/18/26	**DATE SUBMITTED:**	9/21/26
NAME:	SUBALISID, JAPPETH ROYCE B.	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

Hands-on Activity 9.1 Tree ADT

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.



Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. Indicate your code screenshot and comments in table 9-1.

```cpp
#include <iostream>
#include "Tree.h"

int main() {
    Tree<char>* root = new Tree<char>('A');

    root->insert('B');
    root->insert('C');
    root->insert('D');
    root->insert('E');
    root->insert('F');
    root->insert('G');

    root->getChild('D')->insert('H');

    root->getChild('E')->insert('I');
    root->getChild('E')->insert('J');

    root->getChild('F')->insert('K');
    root->getChild('F')->insert('L');
    root->getChild('F')->insert('M');

    root->getChild('G')->insert('N');

    root->getChild('E')->getChild('J')->insert('P');
    root->getChild('E')->getChild('J')->insert('Q');

    return 0;
}
```

```cpp
const int maxChildren = 10;

template <typename T>
class Tree{
    private:
        T data; //data
        Tree<T>* children[maxChildren]; //pointer array to children
    public:
        Tree(T data); //constructor
        void insert(T data); //insert new child
        Tree<T>* getChild(T matchData);
        T getValue();
        void setValue(T newData);
        bool isLeaf();
};
```

```cpp
template <typename T>
Tree<T>::Tree(T newData) {
    data = newData;
    for (int i = 0; i < maxChildren; i++) {
        children[i] = nullptr;
    }
}

template <typename T>
void Tree<T>::insert(T data) {
    for(int i = 0; i < maxChildren; i++) {//loop thru array of children
        if(children[i] == nullptr) {//insert at first nullpointer
            children[i] = new Tree<T>(data);
            return;
        }
    }
    std::cout << "\nThere's no more children slot\n"; //print if didn't return
}
```

```cpp
template <typename T>
Tree<T>* Tree<T>::getChild(T matchData) {
    for(int i = 0; i < maxChildren; i++) {
        if(children[i] != nullptr && children[i]->data == matchData) {
            return children[i];
        }
    }
    std::cout << "Child does not exist.\n";
    return nullptr;
}

template <typename T>
T Tree<T>::getValue() {
    return data;
}

template <typename T>
void Tree<T>::setValue(T newData) {
    data = newData;
}

template <typename T>
bool Tree<T>::isLeaf() {
    for(int i = 0; i < maxChildren; i++) { //loop thru children array
        if(children[i] != nullptr) {//if one isnt null, return false
            return false;
        }
    }
    return true; //if all clear
}
```

Task 2: Complete the following table:

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

Include this completed table as table 9-2 in section 6.

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Task 3: After implementing the code for above, answer the following:

3.1 Given the tree diagram, find the result of the pre-order, post-order and in-order traversal strategies by hand. **Include this as table 9-3 in section 6.**

Pre-Order	A B C D H E I J P Q F K L M G N
Post-Order	B C H D I P Q J E K L M F N G A
In-Order	B A C H D I E P J Q K F L M N G

3.2 Create a function for pre-order, post-order and in-order traversal. Make sure that each function displays an output into the console. Once you have the output, **create and fill table 9-4 in section 6** so that it contains the screenshot of the function, screenshot of the output and your observations. Your observations consist of a comparison between output in #1 and the output of your functions in #2.

```
template <typename T>
void Tree<T>::preOrder() {
    std::cout << data << ' ';
    for(int i = 0; i < maxChildren; i++) {
        if(children[i] != nullptr) {
            children[i]->preOrder();
        }
    }
}

template <typename T>
void Tree<T>::postOrder() {
    for(int i = 0; i < maxChildren; i++) {
        if(children[i] != nullptr) {
            children[i]->postOrder();
        }
    }
    std::cout << data << ' ';
}

template <typename T>
void Tree<T>::inOrder() {
    if(children[0] != nullptr) {
        children[0]->inOrder();
    }
    std::cout << data << ' ';

    for(int i = 1; i < maxChildren; i++) {
        if(children[i] != nullptr) {
            children[i]->inOrder();
        }
    }
}
```

```
Pre-Order Traverse
A B C D H E I J P Q F K L M G N
Post-Order Traverse
B C H D I P Q J E K L M F N G A
In-Order Traverse
B A C H D I E P J Q K F L M N G
```

- **Comparing my initial traversal done by hand with the coded output, they resulted in the same sequence of elements, which means both are correct(or neither).**

3.3. Create a function called findData that will take the following parameter: “CHOICE, KEY”. Choice will determine which traversal method will be used to traverse the tree to find the second parameter KEY. If the KEY is found, the value will be displayed such that: “{KEY} was found!”. Otherwise, no output. **Include your output as table 9-5 in section 6.**

```
template <typename T>
bool Tree<T>::findData(int choice, T findData) {
    bool found = false;

    switch(choice){
        case 1: { //pre order search
            if(data == findData) {
                std::cout << findData << " was found!\n";
                return true;
            }else {
                for(int i = 0; i < maxChildren; i++) {
                    if(children[i] != nullptr && children[i]->findData(1, findData)) {
                        return true;
                    }
                }
            }
            break;
        }
        case 2: { //in order search
            if(children[0] != nullptr && children[0]->findData(2, findData)) {
                found = true;
            }else if(data == findData) {
                std::cout << findData << " was found!\n";
                return true;
            }else {
                for(int i = 1; i < maxChildren; i++) {
                    if(children[i] != nullptr && children[i]->findData(2, findData)) {
                        return true;
                    }
                }
            }
            break;
        }
        case 3: { //post order search
            for(int i = 0; i < maxChildren; i++) {
                if(children[i] != nullptr && children[i]->findData(3, findData)) {
                    found = true;
                    break;
                }
            }
            if(!found && data == findData) {
                std::cout << findData << " was found!\n";
                return true;
            }
            break;
        }
        default:
            std::cout << "\nInvalid choice!\n";
            return false;
    }

    return found;
}
```

```
std::cout << "\nFinding J\n";
root->findData(1, 'J');
std::cout << "\nFinding K\n";
root->findData(1, 'K');
std::cout << "\nFinding A\n";
root->findData(1, 'A');
```

```
Finding J
J was found!

Finding K
K was found!

Finding A
A was found!
```

3.4. Add a new leaf node called which will contain the data value of "O". Make this node the child of nodeG. Perform the findData function you created in #3, what can you observe about the output? **Include your output and answers as table 9-6 in section 6**

```
std::cout << "\nInserting O\n";
root->getChild('G')->insert('O');
std::cout << "\nFinding O\n";
root->findData(1, 'O');
```

```
Inserting O

Finding O
O was found!
```

- **We can see with the code and terminal output that the insertion and searching were both successful. This is because I made the tree to have branches that's able to contain a maximum of 10 branches. To add, the searching techniques all go through each node once, therefore, the time complexity for searching is O(n).**

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

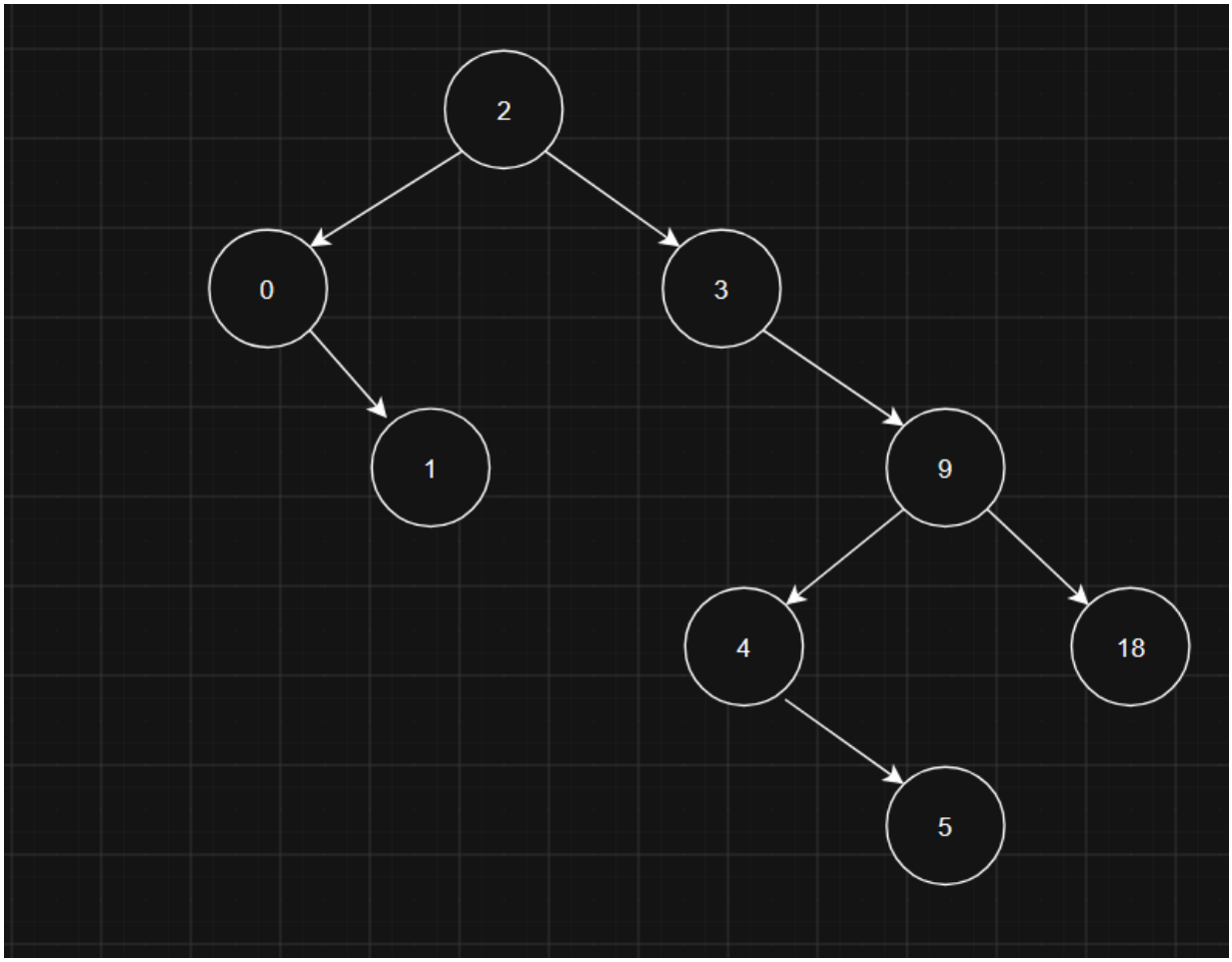
ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

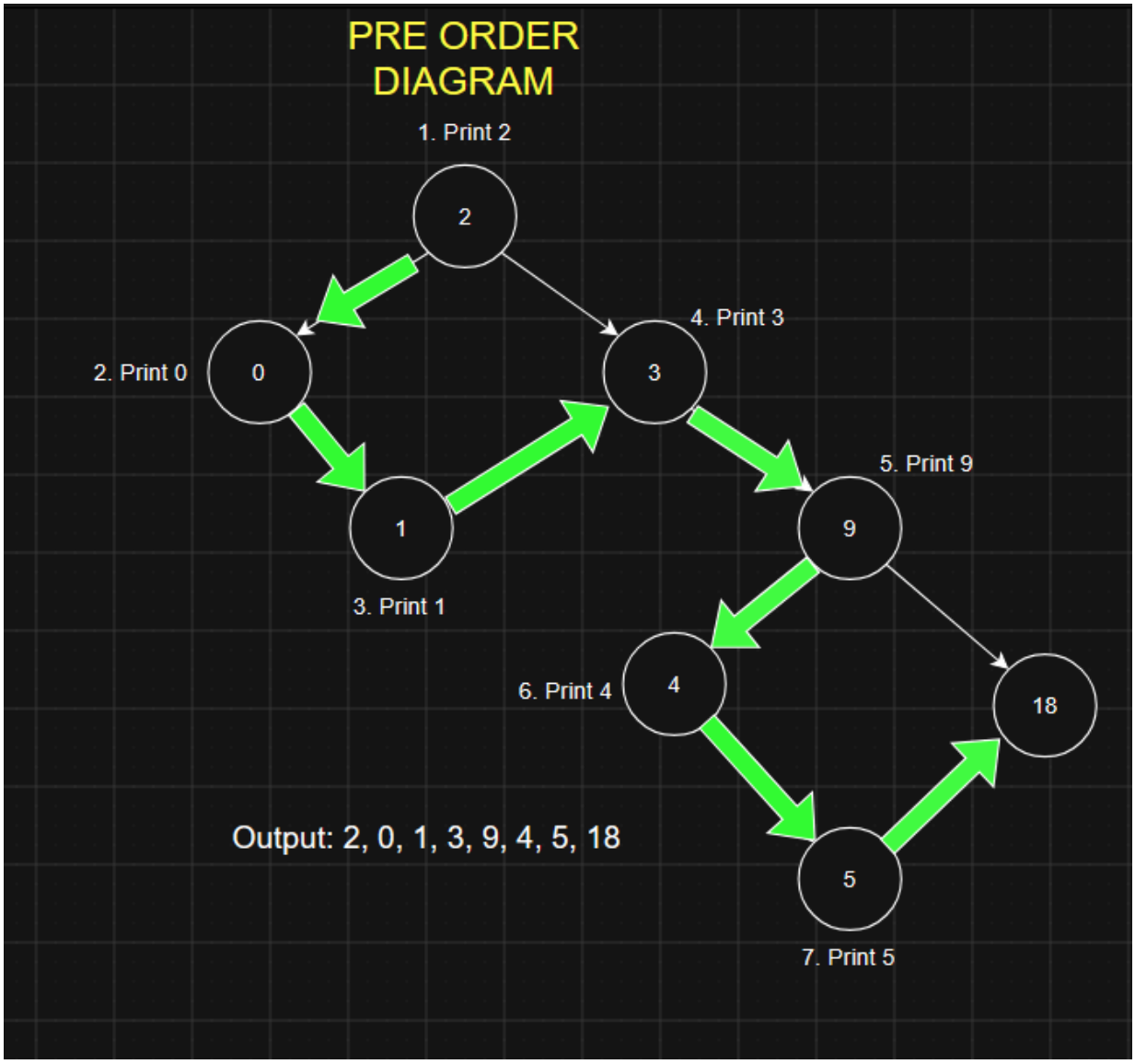
```
int main() {
    BinarySearchTree<int> root(2); //root node

    root.insert(3);
    root.insert(9);
    root.insert(18);
    root.insert(0);
    root.insert(1);
    root.insert(4);
    root.insert(5);
```

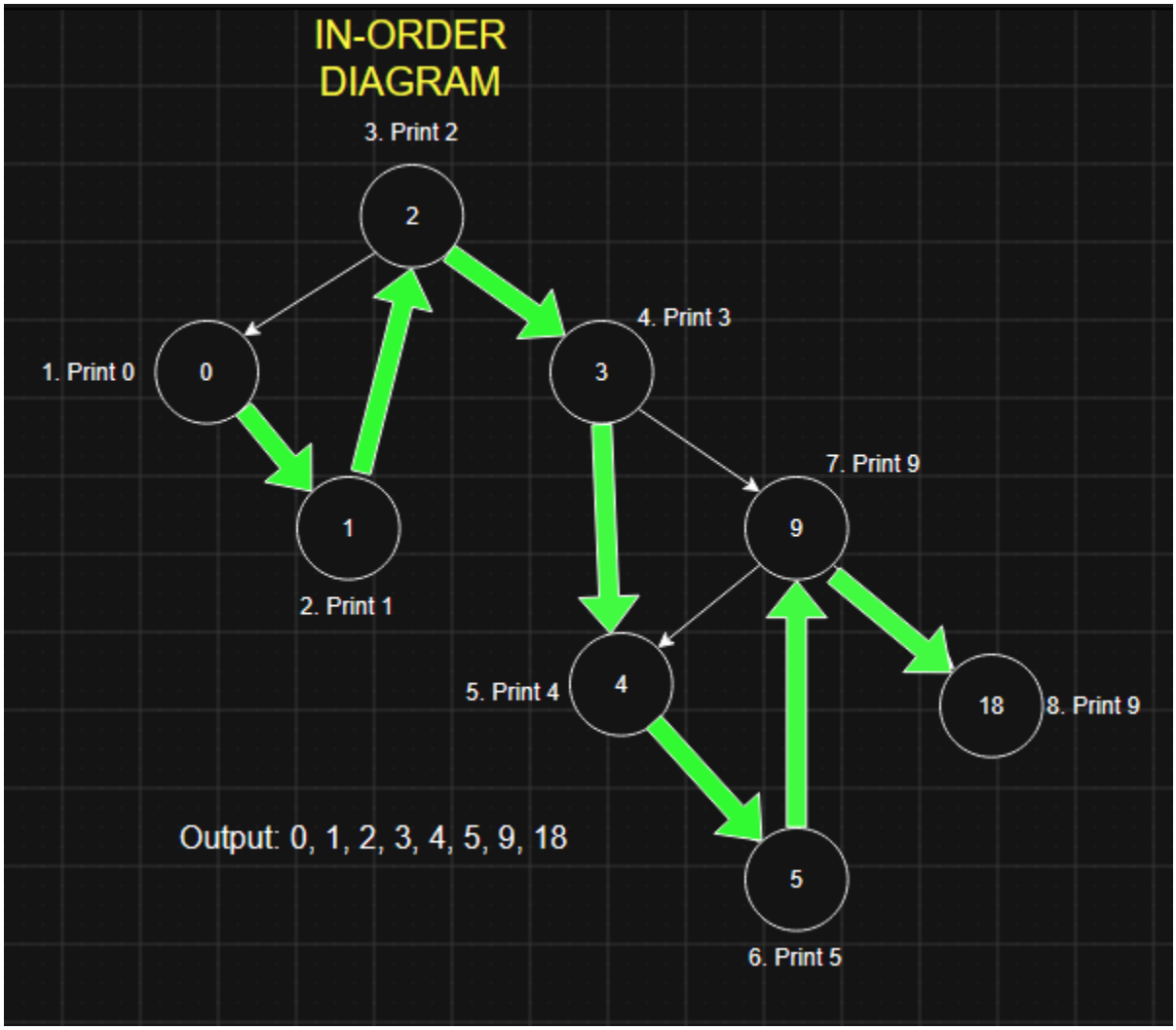
Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.



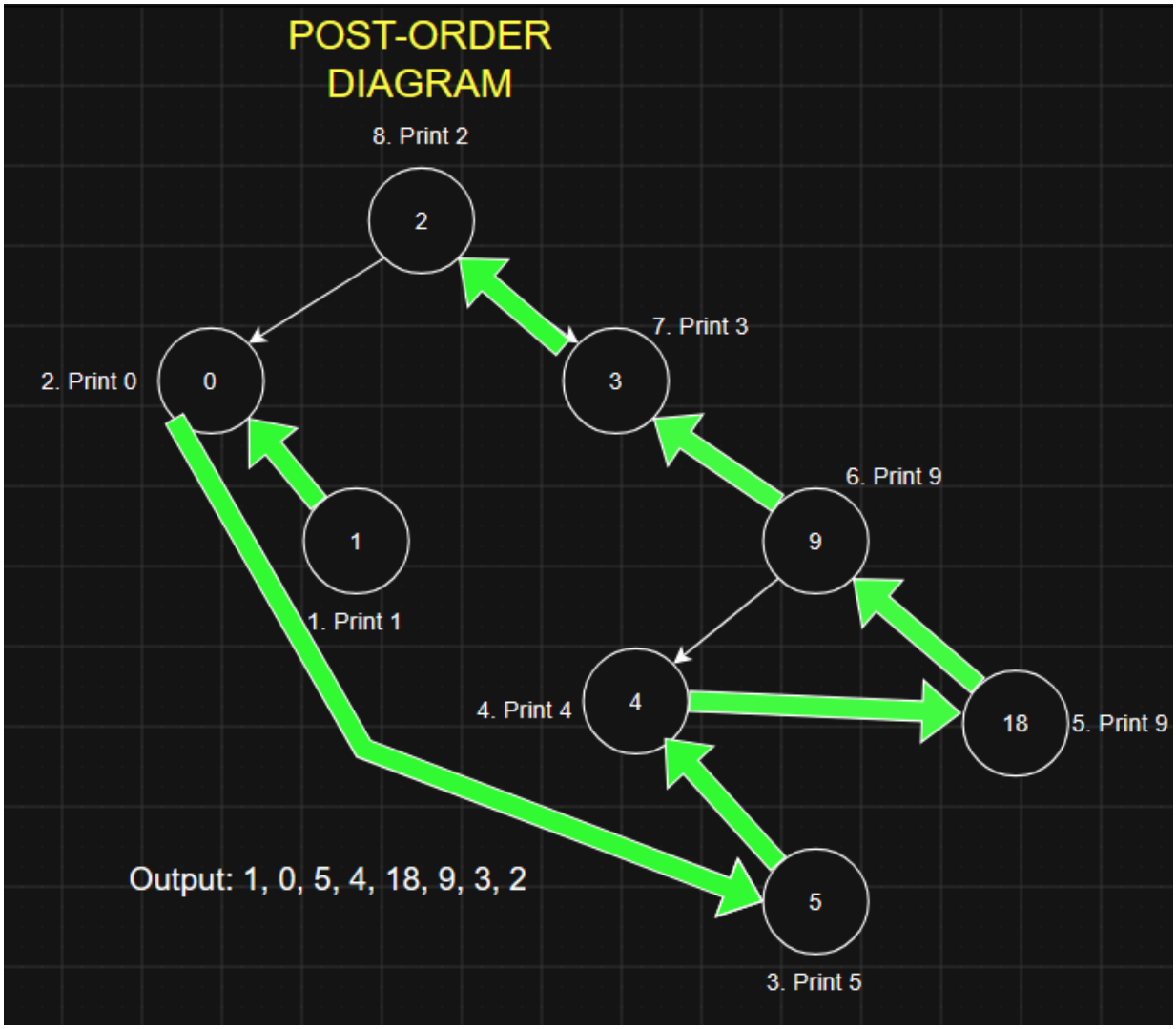
Tree Diagram



Pre-order Traversal



In-order Traversal



Post-order Traversal

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
Pre-order: 2 0 1 3 9 4 5 18
In-order: 0 1 2 3 4 5 9 18
Post-order: 1 0 5 4 18 9 3 2
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

```
template <typename T>
void BinarySearchTree<T>::preOrder() {
        std::cout << key << ' '; //print key

        if(left != nullptr) { //recurse on left until there's no left
            left->preOrder();
        }

        if(right != nullptr) {//check right if left is done
            right->preOrder();
        }
}
```

```
Pre-order: 2 0 1 3 9 4 5 18
```

In-order Traversal

```
template <typename T>
void BinarySearchTree<T>::inOrder() {
    if(left != nullptr) { //recurse to the left until left is null
        left->inOrder();
    }

    std::cout << key << ' ';

    if(right != nullptr) {
        right->inOrder();
    }
}
```

```
In-order: 0 1 2 3 4 5 9 18
```

Post-order Traversal

```
template <typename T>
void BinarySearchTree<T>::postOrder() {
    if(left != nullptr) { //recurse to the left until left is null
        left->postOrder();
    }

    if(right != nullptr) {
        right->postOrder();
    }

    std::cout << key << ' ';
}
```

```
Post-order: 1 0 5 4 18 9 3 2
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Finding errors to be fixed**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **After implementing the core idea of the functions to be used, I used it to find where the errors are whenever I encounter a bug that I get stuck in.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.